

CS121 Data Structures in C

08 *Trees*

Outline

- Introduction of trees
 - ♦ Some terminologies
 - ♦ Implementation and example
- Tree traversal
 - ♦ Preorder
 - ♦ Postorder
- Binary tree
- Binary search tree and its operations

Introduction

- Linear access time($O(N)$) of linked lists is prohibitive.
 - ♦ It takes $O(N)$ time for most of the operations on linked lists, e.g. search for a specific value, insert a value into the linked list for sorting.
 - ♦ Does there exist any simple data structure for which the running time of most operations (search, insert, delete) is $O(\log N)$?
- Trees
 - ♦ Basic concepts
 - ♦ Tree traversal
 - ♦ Binary tree
 - ♦ Binary search tree and its operations

Trees

- A tree is a collection of nodes
 - ♦ The collection can be empty.
 - ♦ (recursive definition) If not empty, a tree consists of a distinguished node r (the **root**), and **zero or more** nonempty **subtrees** $T_1, T_2, \dots, T_k$, each of whose roots are connected by a directed **edge** from r .
- Tree is a recursive data type.

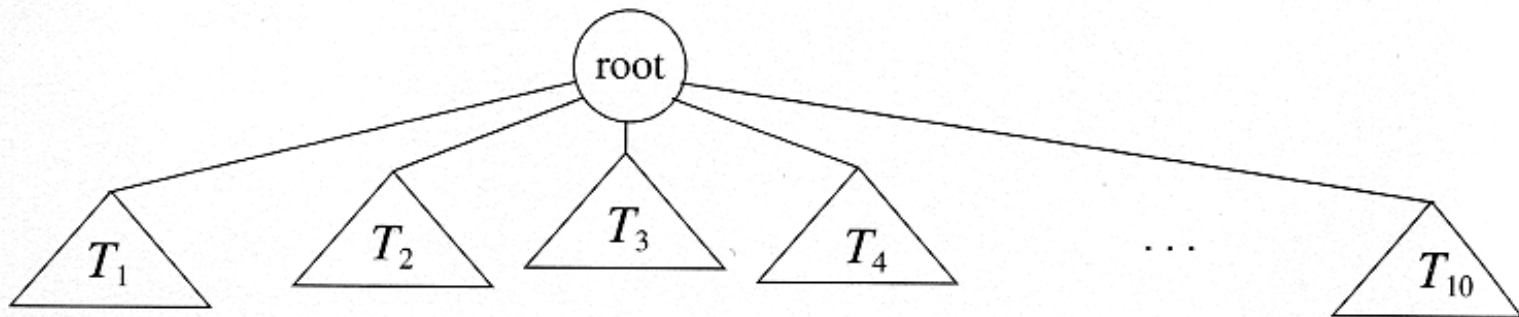
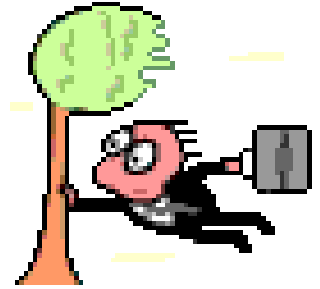


Figure 4.1 Generic tree

Some Terminologies

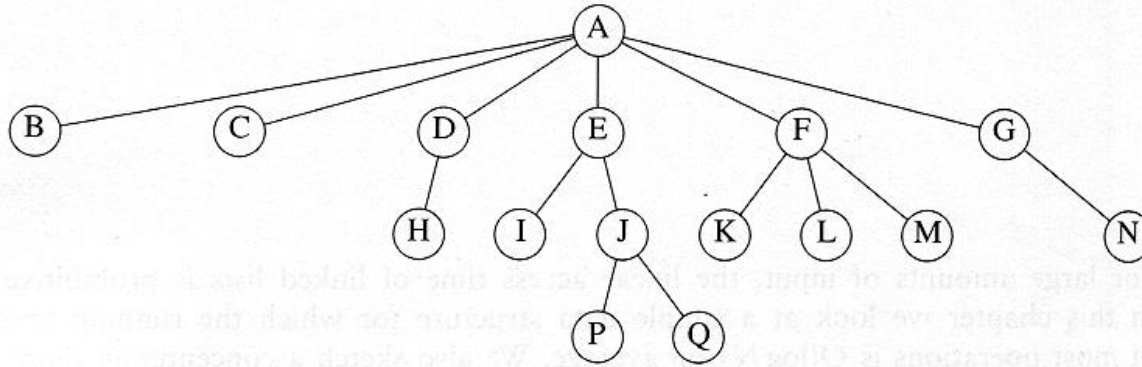


Figure 4.2 A tree

- *Child* and *Parent*
 - ♦ Every node except the root has one parent.
 - ♦ A node can have zero or more children.
- *Leaves*
 - ♦ Leaves are nodes with no children.
- *Sibling*
 - ♦ nodes with the same parent.

Some Terminologies

- *Path* from node n_1 to n_k
 - ♦ A sequence of nodes $n_1, n_2, \dots, n_k$ such that n_i is the parent of n_{i+1} for $1 \leq i < k$.
- *Length of a path*
 - ♦ number of edges on the path, namely $k-1$
- *Depth of a node*
 - ♦ length of the unique path from the root to that node. The root is at depth 0.
- *Height of a node*
 - ♦ length of the longest path from that node to a leaf.
 - ♦ all leaves are at height 0.
- The height of a tree = the height of the root
= the depth of the deepest leaf
- *Ancestor* and *descendant*
 - ♦ If there is a path from n_1 to n_2 .
 - ♦ n_1 is an ancestor of n_2 , n_2 is a descendant of n_1 , if $n_1 \neq n_2$.
 - ♦ *Proper ancestor* and *proper descendant*.

Example

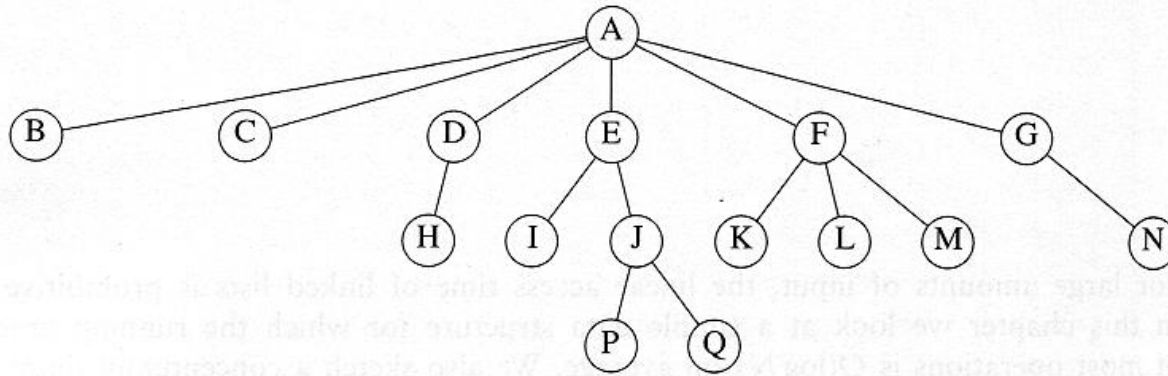
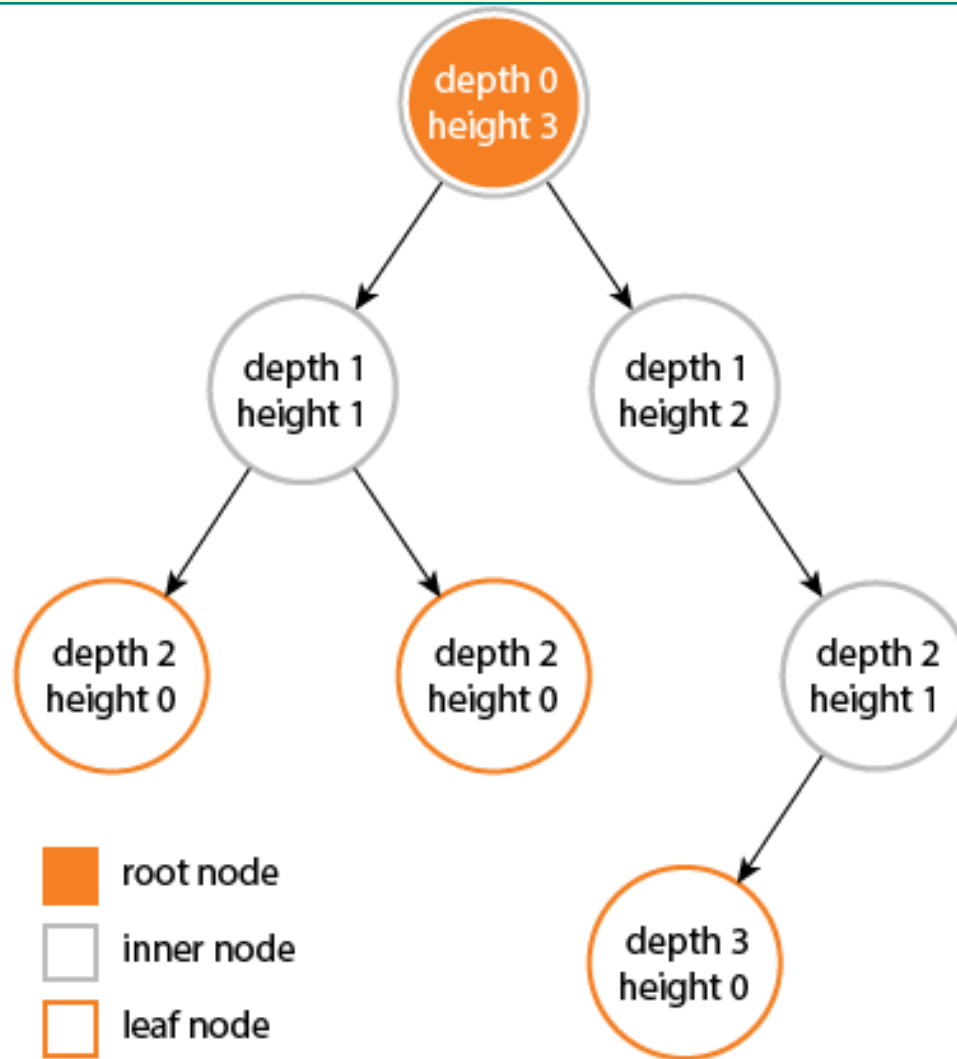


Figure 4.2 A tree

- The root is A; the leaves are B,C,H,I,P,Q,K,L,M,N
- F – parent: A ; children: K, L, M (K, L and M are siblings)
- Path from **A** to **P**: A,E,J,P
- Depth of **P**: 3
- Height of **E**: 2 (the longest path for E to the leaf)
- Height of the **tree**: 3

Example



<https://stackoverflow.com/questions/2603692/what-is-the-difference-between-tree-depth-and-height>

Application Example for Tree

- The directory structure of operating system
the typical directory in Unix file system

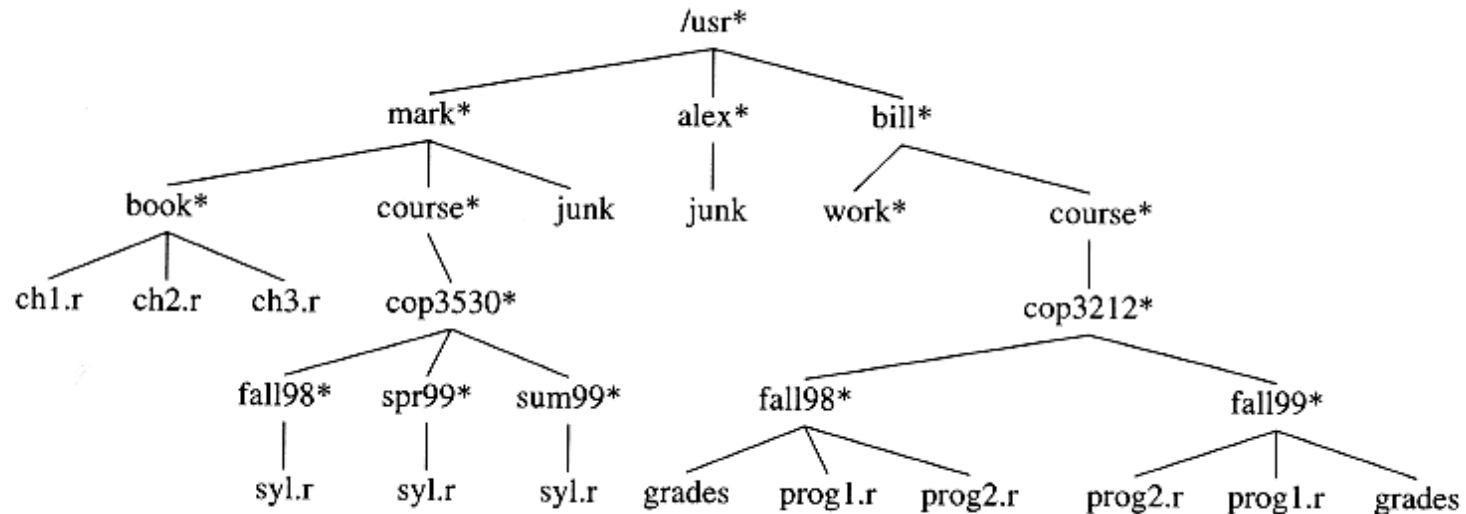


Figure 4.5 UNIX directory

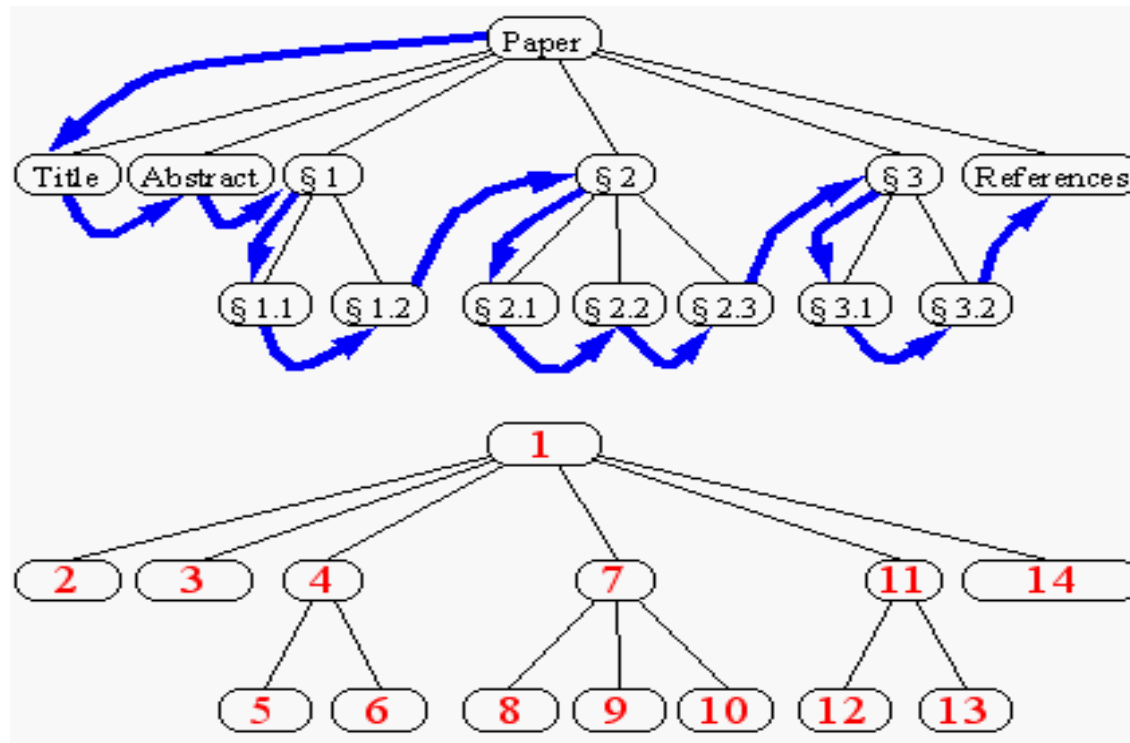
- The root of the directory: /usr
- Three children: mark, alex, bill
- Filename: /usr/mark/book/ch1.r : each “/” after the first indicates an edge. It is the full pathname.

Tree Traversal

- Tree traversal is used to print out the data in a tree in a certain order.
- Strategy
 - ◆ Preorder traversal
 - The work at a node is performed before its children are processed.
 - ◆ Postorder traversal
 - The work at a node is performed after its children are processed.

Preorder Traversal

- Preorder traversal
 - ♦ First, visit the root.
 - ♦ Then, recursively traverse every sub-tree of the root.



Postorder Traversal

- Postorder traversal
 - ◆ First, recursively traverse every sub-tree of the root;
 - ◆ Then, visit the root.

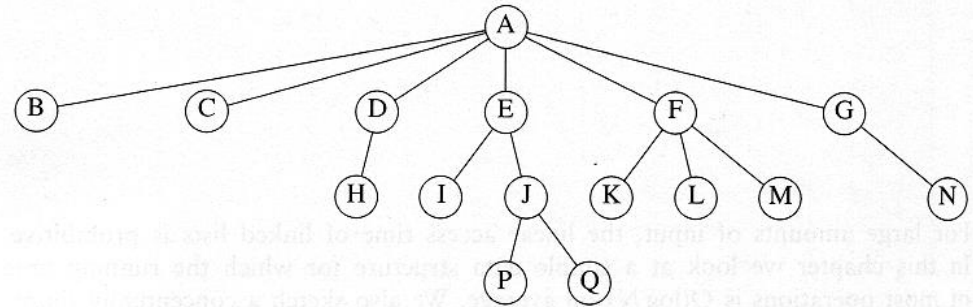
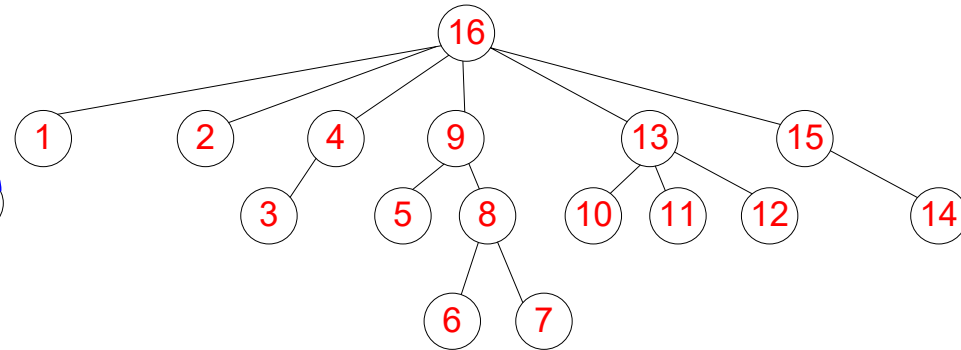
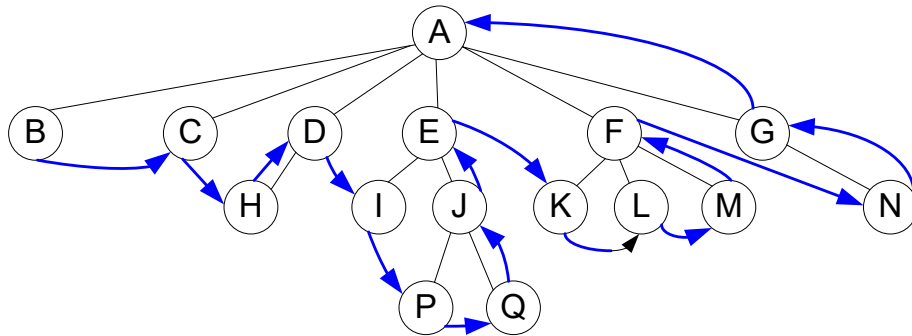


Figure 4.2 A tree

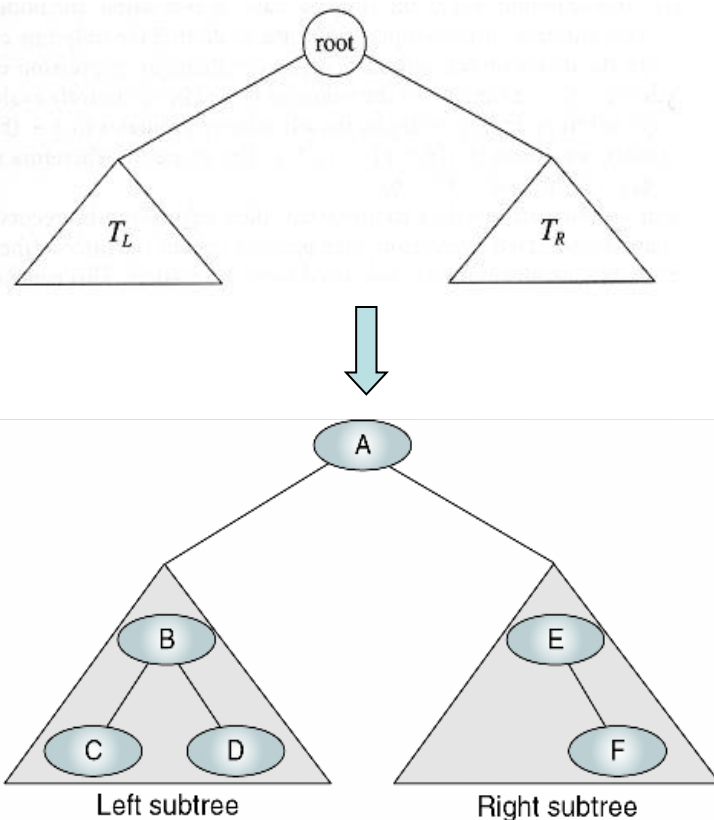


BCHDIPQJEKLMFNGA

Binary Tree

Binary Tree

- A special class of trees:
 - ♦ Either empty;
 - ♦ Or, a node with exactly 2 sub (binary)-tree (may all be empty). The 2 sub-trees are called *left* and *right* sub-tree. It is an ordered tree.



**Generic
binary tree**

• Binary tree is a special case of tree, however, it can represent all types of tree.

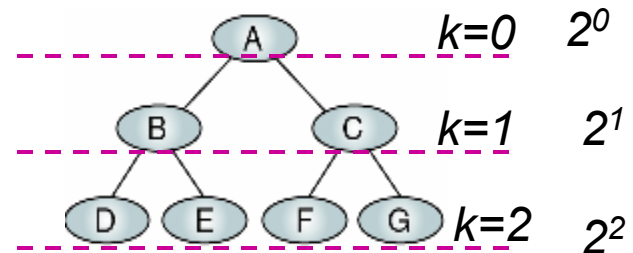
Some Properties of Binary Tree

- There are at most 2^i nodes at depth i .
- A binary tree of depth k has at most $2^{k+1}-1$ nodes.

$$\diamond \sum_{depth=0}^k \text{node} = f(k) = 2^0 + 2^1 + 2^2 + \dots + 2^k$$

$$2f(k) = 2^1 + 2^2 + 2^3 + \dots + 2^{k+1},$$

$$\diamond f(k) = 2f(k) - f(k) = 2^{k+1} - 2^0.$$

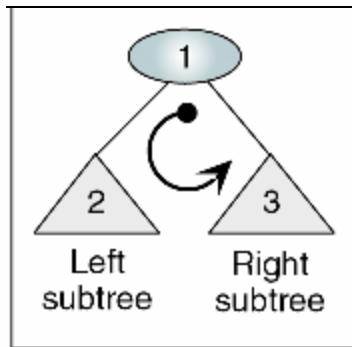


Complete Binary Tree

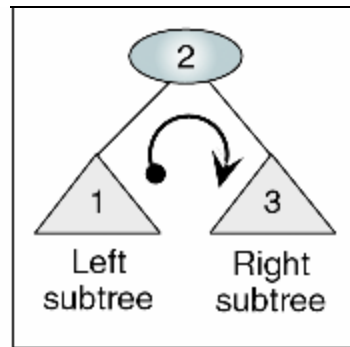
- A binary tree is a *full binary tree* if and only if:
 - ♦ Each non-leaf node has exactly two child nodes
 - ♦ All leaf nodes have identical path length
- It is called full since all possible node slots are occupied
 - ♦ A *full binary tree* of depth k has how many nodes? $2^{k+1}-1$
 - ♦ A *full binary tree* of depth k has how many leaves? 2^k
 - ♦ A *full binary tree* of N nodes has depth of $O(\log N)$.
- A *complete binary tree* of depth k is either a full binary tree, or only level k isn't full and the leaves at depth k are located to the left.
 - ♦ All nodes at depth $k-1$ or less are full.

Binary Tree Traversal

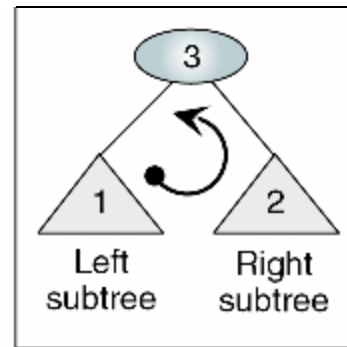
- Strategy
 - ♦ Preorder traversal
 - ♦ Postorder traversal
 - ♦ Inorder traversal



(a) Preorder traversal



(b) Inorder traversal



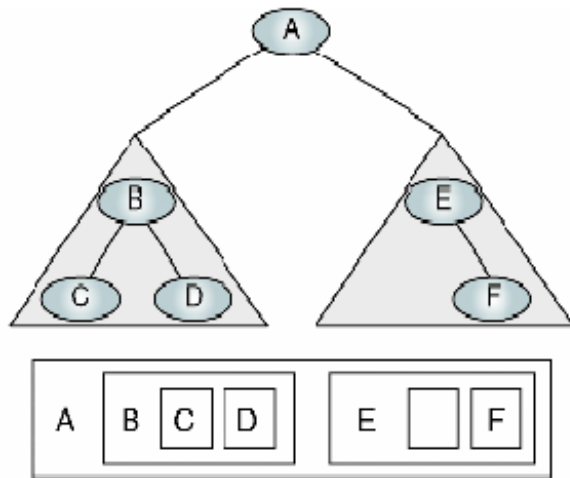
(c) Postorder traversal

(Data Structures: A Pseudocode Approach with C)

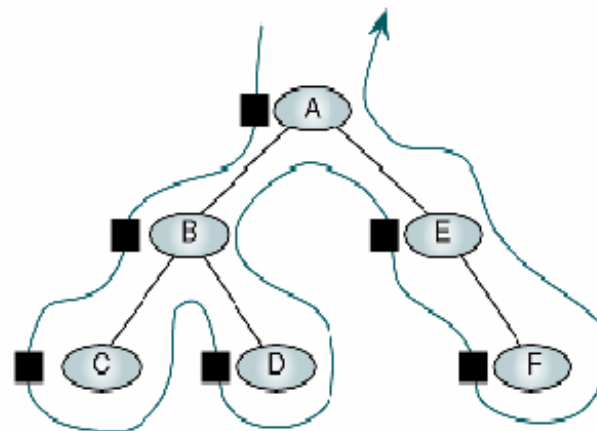
Preorder Traversal

- In the **preorder** traversal, the **root** is processed first, before **its subtrees**.
- The root goes before its subtrees.

Preorder Traversal—A B C D E F



(a) Processing order



(b) "Walking" order

(Data Structures: A Pseudocode Approach with C)

Implementation Preorder Traversal

- Binary tree

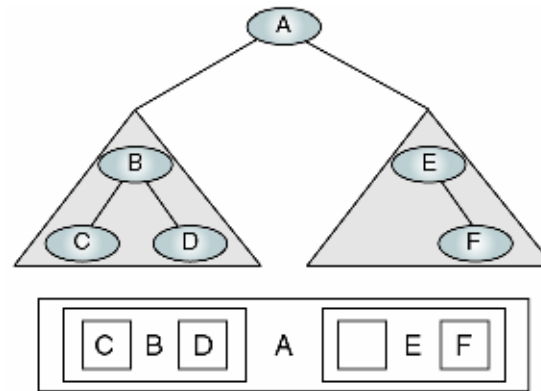
```
typedef int ElementType;
struct BTreeNode;
struct BTreeNode
{
    ElementType      Element;
    struct BTreeNode * Left;
    struct BTreeNode * Right;
};
typedef struct BTreeNode  BTreeNode;
-----
```

- Preorder traversal

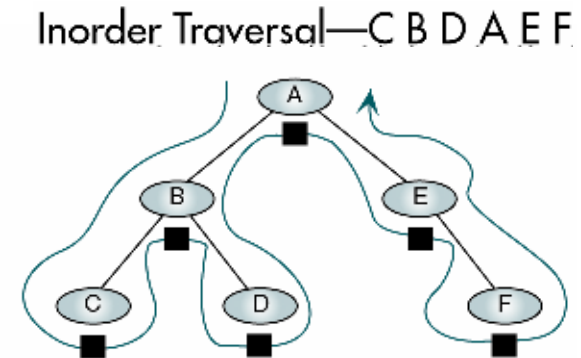
```
void
Preorder(BTreeNode *node)
{
    if (node != NULL)
    {
        printf("%d", node->Element);
        preorder(node->Left);
        preorder(node->Right);
    }
}
```

Inorder Traversal and Implementation

- In the **inorder** traversal, the **root** is processed between its **subtrees**.



(a) Processing order



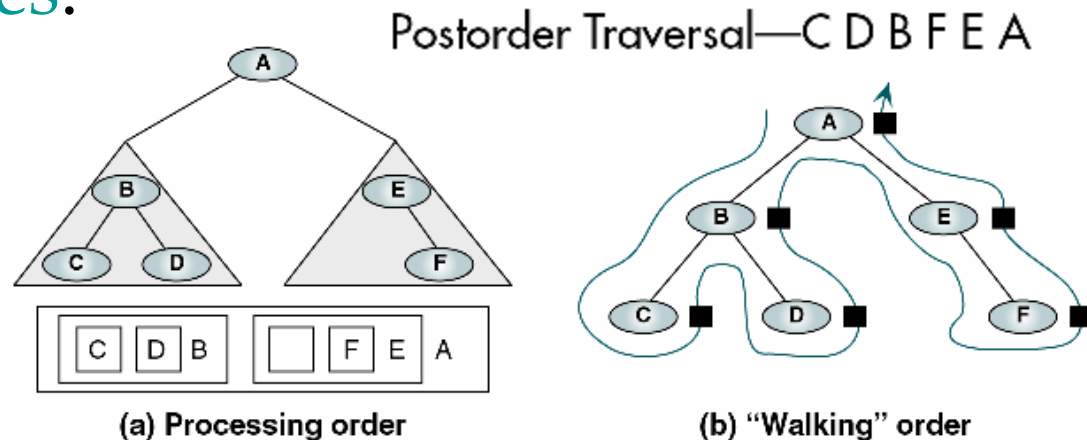
(b) "Walking" order

- Implementation

```
void
Inorder(BTNode *node)
{
    if (node != NULL)
    {
        Inorder(node->Left);
        printf("%d", node->Element);
        Inorder(node->Right);
    }
}
```

Postorder Traversal

- In the **postorder** traversal, the **root** is processed after its **subtrees**.



- Implementation

```
void
Postorder (BTNode    *node)
{
    if (node != NULL)
    {
        Postorder (node->Left) ;
        Postorder (node->Right) ;
        printf ("%d", node->Element) ;
    }
}
```

Example: Binary Tree Traversal

- Preorder traversal
 - ♦ N, L, R
 - ♦ **Prefix** expression
 - ++a*bc*+*defg
- Postorder traversal
 - ♦ L, R, N
 - ♦ **Postfix** expression
 - abc*+de*f+g*+
- Inorder traversal
 - ♦ L, N, R
 - ♦ **Infix** expression
 - a+b*c+d*e+f*g

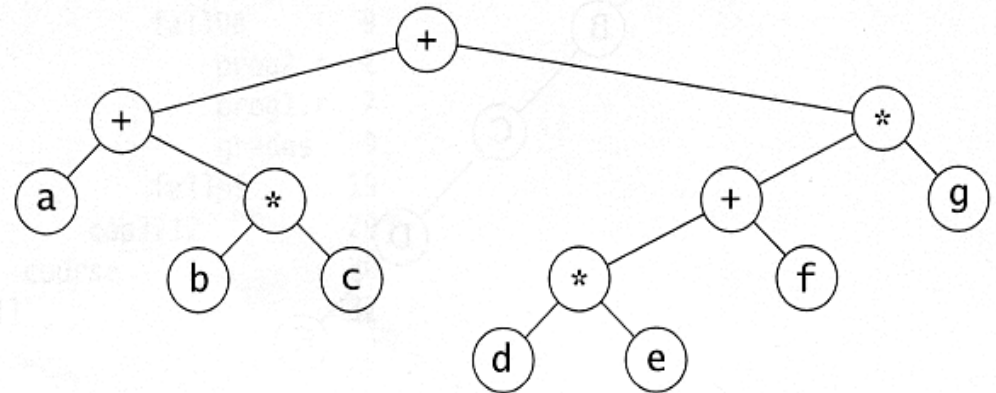


Figure 4.14 Expression tree for $(a + b * c) + ((d * e + f) * g)$

Assignment 07

- Write a recursive function

```
int leaf( BTreeNode *t, int n )
```

to calculate the number of leaf nodes in a binary tree.

Each binary tree node is defined as follows:

```
typedef int ElementType;
struct BTreeNode;
struct BTreeNode
{
    ElementType      Element;
    struct BTreeNode * Left;
    struct BTreeNode * Right;
};
typedef struct BTreeNode BTreeNode;
```

- Submit it to Moodle.

Binary Search Tree (BST)

Binary Search Tree

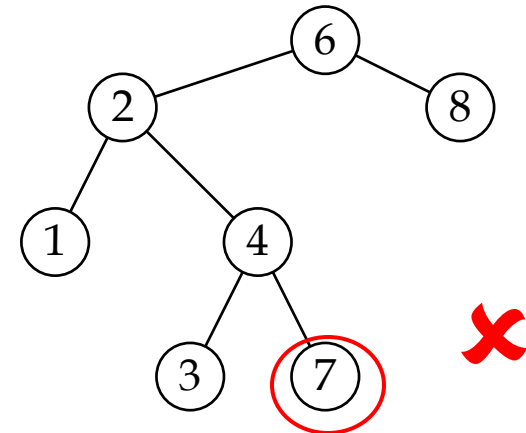
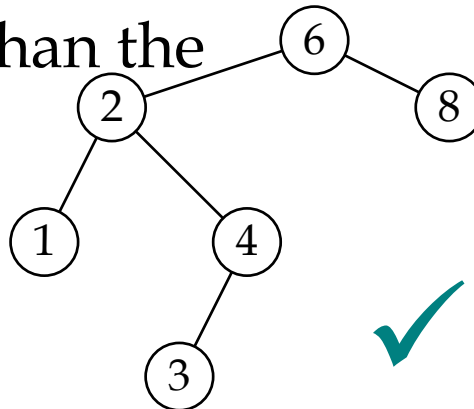
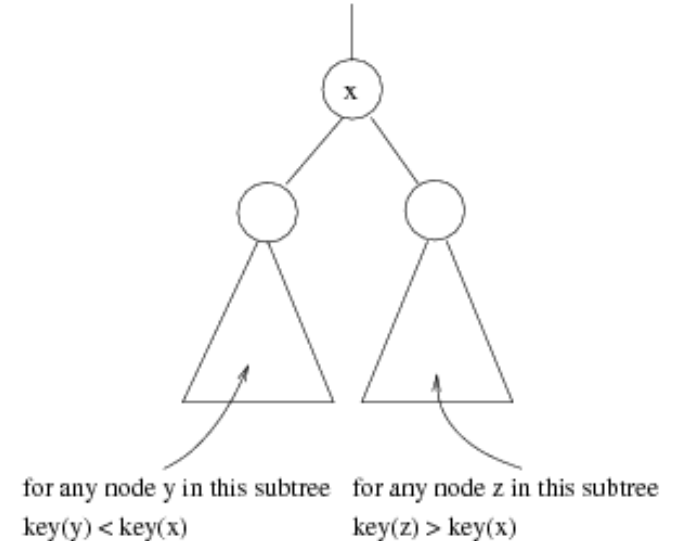
- A data structure for efficient search, insertion and deletion.

- Property of binary search tree

For every node X ,

- ♦ all keys in its **left subtree** are **smaller** than the key value in X ; and
- ♦ all keys in its **right subtree** are **larger** than the key value in X .

And each subtree is itself a binary search tree.



Operations of BST

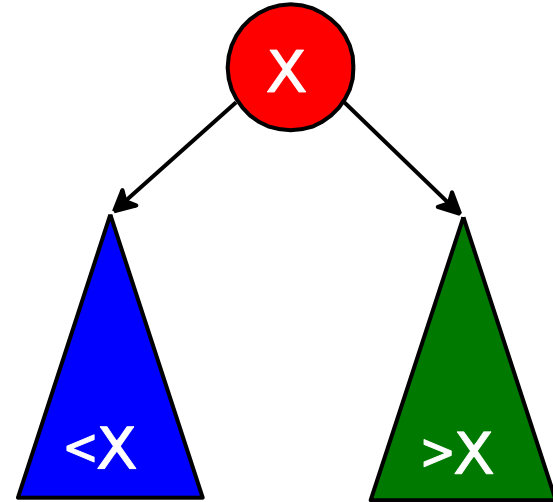
- Find
 - ♦ Find the key value X in BST
 - ♦ Find the smallest and largest values in BST
- Insert
 - ♦ Insert X into BST
- Deletion
 - ♦ Delete X from BST

Find Key X

- The main application of binary search trees is to search for a node with a given key value X .
- According to the properties of binary search trees, we may find a node very quickly:
 - ♦ If the tree is empty, we return NULL (not found).
 - ♦ If the root node has key X , we simply return it.
 - ♦ If X is less than the root key, the node to find must be in the left sub-tree. We can recursively search for it.
 - ♦ If X is greater than the root key, we can recursively search for it in the right sub-tree.
- The searching advances a level in each step, so the number of searching steps will not be more than the depth of the tree.

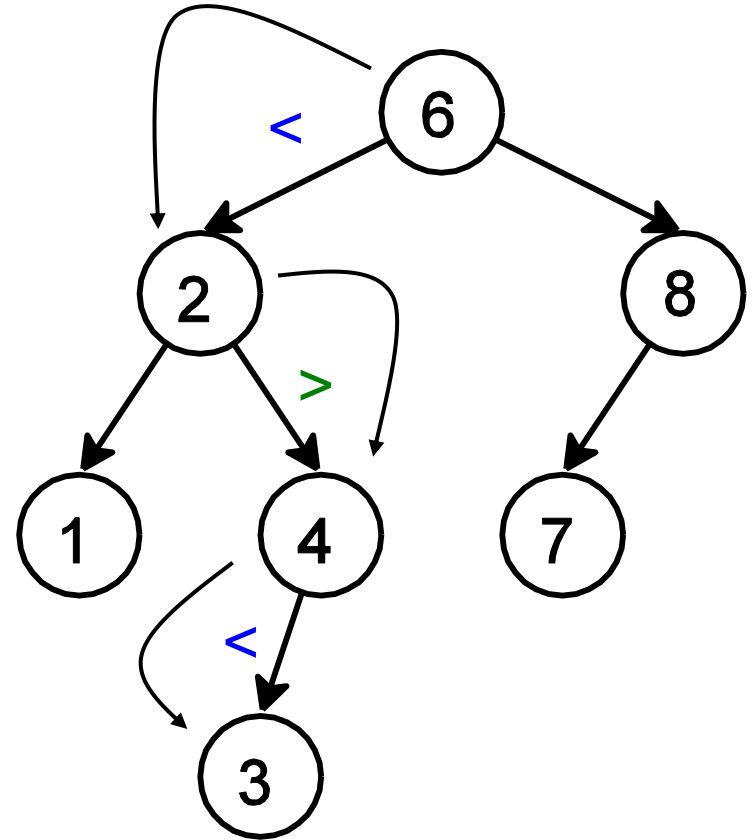
Find Key X

- To look for a node with a certain Key
- $\text{Left} < \text{Node} < \text{Right}$
- Recursively do:
 1. Compare (Key,node)
 2. $\text{Key} < \text{node}$:
Go left
 3. $\text{Key} > \text{node}$:
Go right



Example: Find the Key 3

- node 6, **Key** < 6
- Go **left**, node 2, **Key** > 2
- Go **right**, node 4, **Key** < 4
- Go **left**, node 3, **Key** == 3
- O(1) at each level
- Find a key
 - ♦ O(depth) if found
 - ♦ O(height) if not found
- Time complexity=O(height)



Implementation: Find

- Firstly, initialization of BST
 - ◆ Make an empty BST

```
SearchTree
MakeEmpty( SearchTree T )
{
    if( T != NULL )
    {
        MakeEmpty( T->Left );
        MakeEmpty( T->Right );
        free( T );
    }
    return NULL;
}
```

```
typedef int ElementType;

struct TreeNode;
typedef struct TreeNode *Position;
typedef struct TreeNode *SearchTree;

struct TreeNode
{
    ElementType Element;
    SearchTree Left;
    SearchTree Right;
};
```

Implementation: Find (cont.)

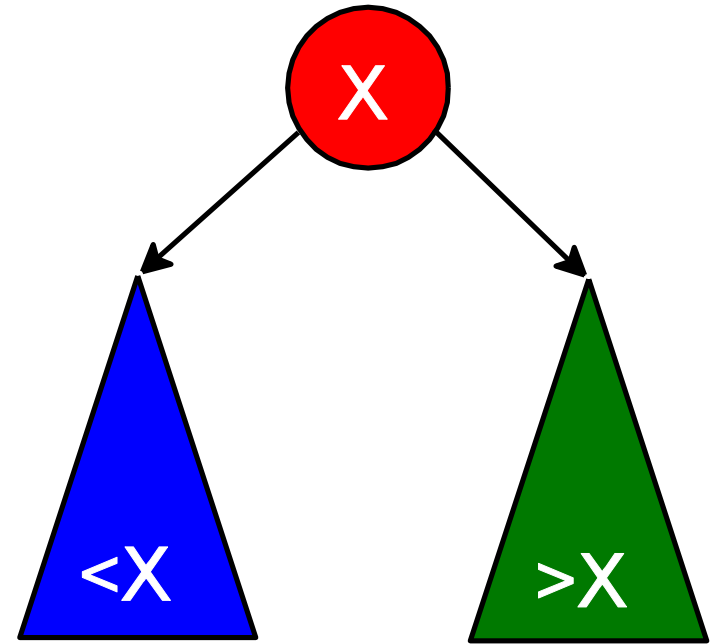
- **Find** X: return a pointer to the node that has key X, or NULL if there is no such node.

Position

```
Find( ElementType X, SearchTree T )
{
    if( T == NULL )
        return NULL;
    if( X < T->Element )
        return Find( X, T->Left );
    else
        if( X > T->Element )
            return Find( X, T->Right );
        else
            return T;
}
```

Find Min/Find Max

- To find the smallest/largest value in BST
- $\text{Left} < \text{Node} < \text{Right}$
- Min = Left most node
 - ♦ Recursively go left
 - Until no more left child
- Max = Right most node
 - ♦ Recursively go right
 - Until no more right child



Find Min/Find Max

- **FindMin/FindMax**: return the node containing the smallest (largest) key in the tree.
- Algorithm: Start at the root and go **left** (**right**) as long as there is a **left** (**right**) child. The stopping point is the **smallest** (**largest**) element.

```
Position
FindMin( SearchTree T )
{
    if( T == NULL )
        return NULL;
    else
        if( T->Left == NULL )
            return T;
        else
            return FindMin( T->Left );
}
```

Recursively

```
Position
FindMax( SearchTree T )
{
    if( T != NULL )
        while( T->Right != NULL )
            T = T->Right;

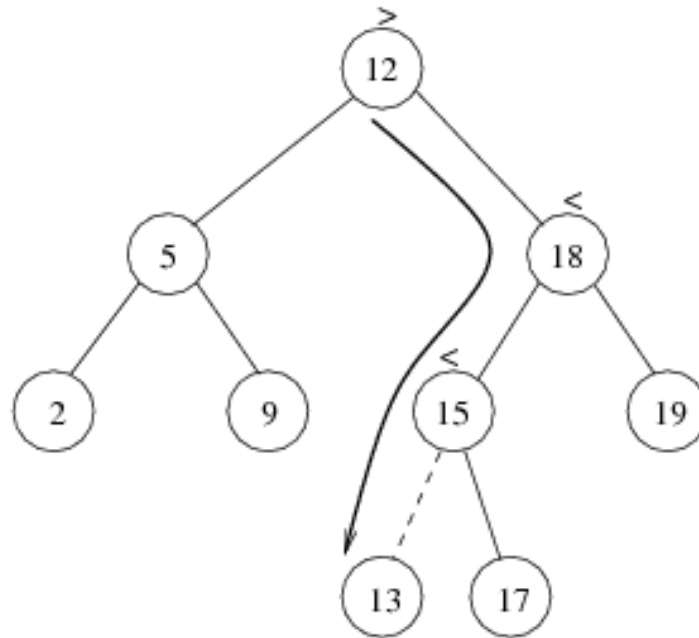
    return T;
}
```

Non-recursively

- Time complexity = $O(\text{height of the tree})$

Insertion

- Proceed down the tree as you would with a **find**.
- If X is found, do nothing.
- Otherwise, insert X at the last spot on the path traversed.



- Time complexity = $O(\text{height of the tree})$

Implementation: Insertion

if tree is empty

create a root node with the new key

else

compare key with the top node

if **key < node key**

*compare the key with the left subtree
(recursively):*

if until the last one

add key to the left subtree

else if **key > node key**

*compare the key with the right
subtree (recursively):*

if until the last one

add key in the right subtree

```
SearchTree
Insert( ElementType X, SearchTree T )
{
    if( T == NULL )
    {
        /* Create and return a one-node tree */
        T = malloc( sizeof( struct TreeNode ) );
        if( T == NULL )
            FatalError( "Out of space!!!" );
        else
        {
            T->Element = X;
            T->Left = T->Right = NULL;
        }
    }
    else
    if( X < T->Element )
        T->Left = Insert( X, T->Left );
    else
    if( X > T->Element )
        T->Right = Insert( X, T->Right );
    /* Else X is in the tree already; we'll do nothing */

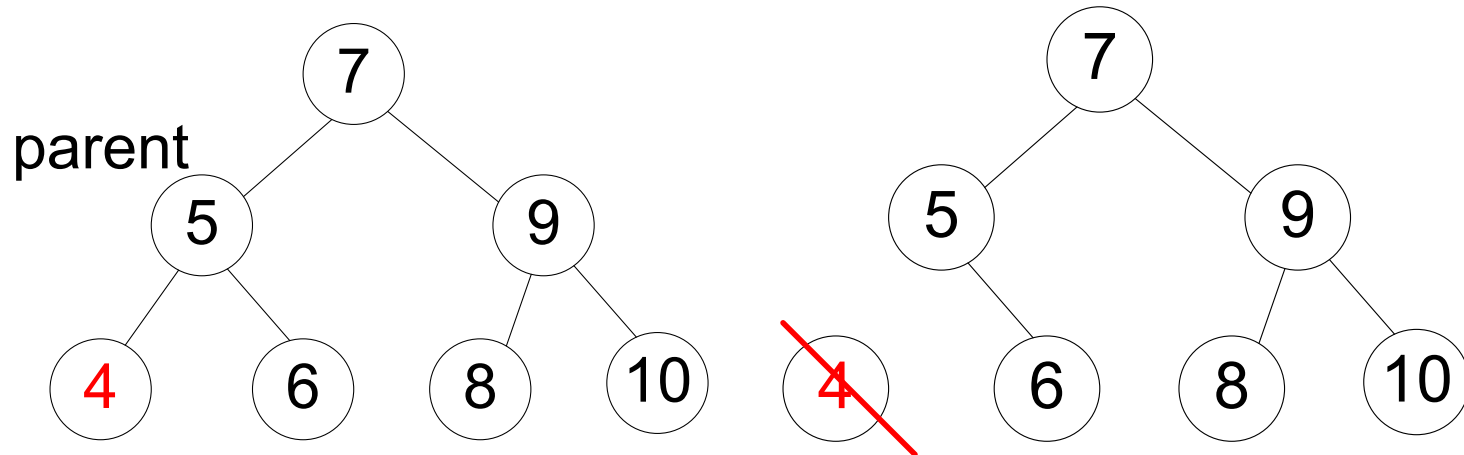
    return T; /* Do not forget this line!! */
}
```

Deletion

- When we delete a node, we need to consider how we take care of the children of the deleted node.
 - ♦ This has to be done such that the property of the binary search tree is maintained.

Example: Deletion

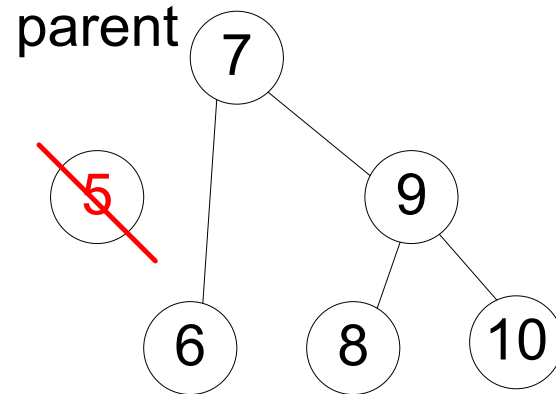
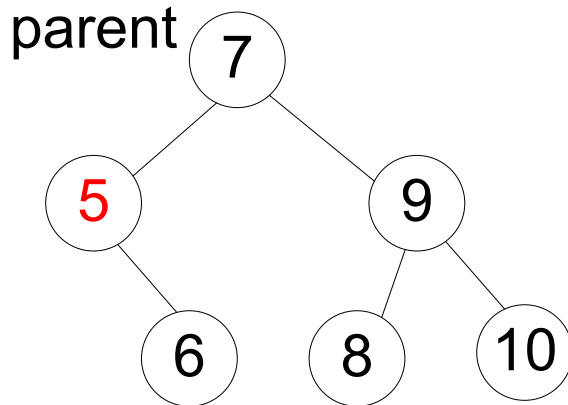
- Case 1: the node is a leaf
 - ♦ Deleting 4
 - ♦ Replace the link in the parent with NULL



Example: Deletion

- Case 2: the node has one child
 - ♦ Adjust a pointer from the parent to bypass that node.
 - Case 2.1: the node has no left child
 - link the parent of the node to the right (non-empty) subtree

Deleting 5



Example: Deletion

- Case 2: the node has one child
 - ♦ Adjust a pointer from the parent to bypass that node.
 - Case 2.2: the node has no right child
 - link the parent of the node to the left (non-empty) subtree

Deleting 4

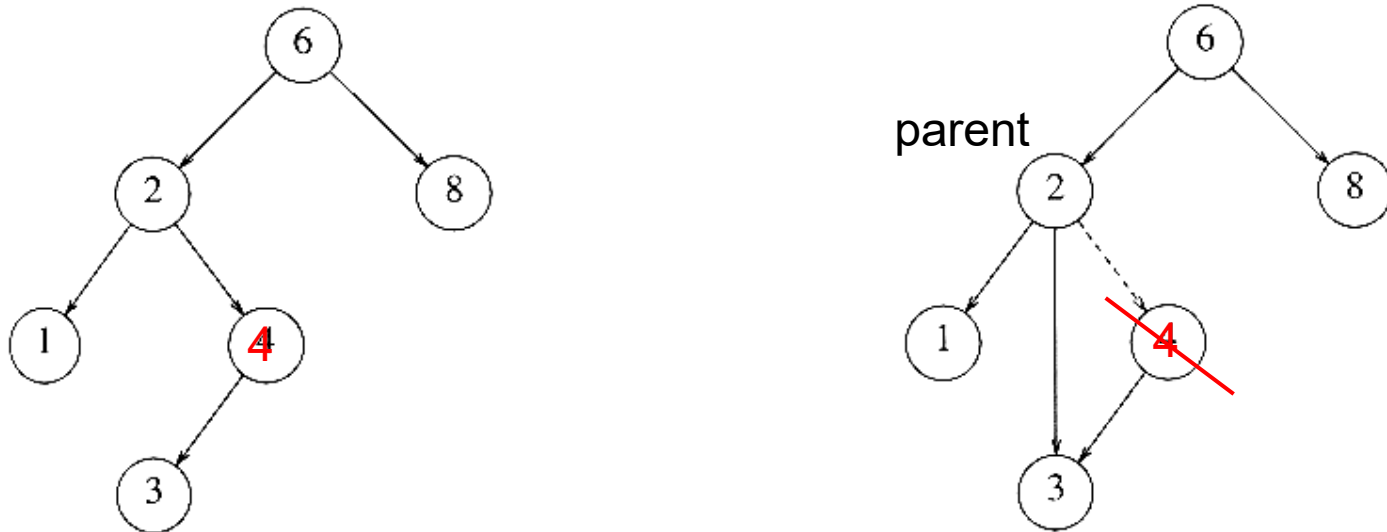


Figure 4.24 Deletion of a node (4) with one child, before and after

Example: Deletion

- Case 3: the node has two children
 - ♦ Replace the key of that node with the minimum element at the right subtree.
 - ♦ Delete that minimum element
 - There is either no child or only right child because if it has a left child, that left child would be smaller and would have been chosen. So invoke case 1 or case 2.

Deleting 2

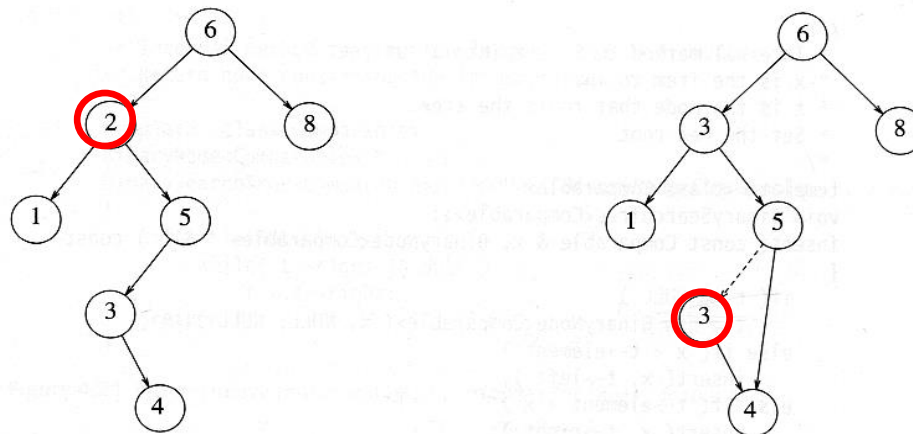


Figure 4.25 Deletion of a node (2) with two children, before and after

- Time complexity = $O(\text{height of the tree})$

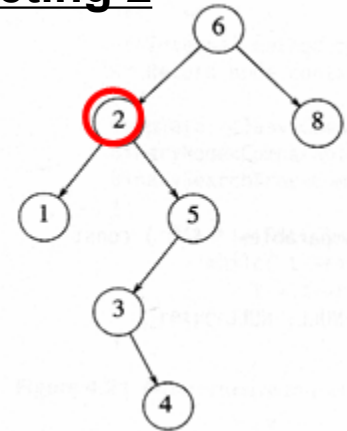
Implementation: Deletion

```
SearchTree
Delete( ElementType X, SearchTree T )
{
    Position TmpCell;
```

To find X

```
    if( T == NULL )
        Error( "Element not found" );
    else
        if( X < T->Element ) /* Go left */
            T->Left = Delete( X, T->Left );
        else
            if( X > T->Element ) /* Go right */
                T->Right = Delete( X, T->Right );
            else /* Found element to be deleted */
                if( T->Left && T->Right ) /* Two children */
                {
                    .....
                }
            }
        }
```

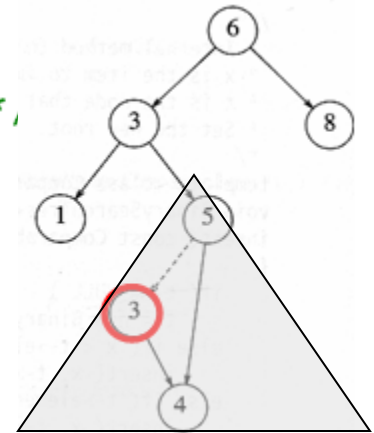
Deleting 2



Implementation: Deletion

```
if( T->Left && T->Right ) /* Two children */
{
    /* Replace with smallest in right subtree */
    TmpCell = FindMin( T->Right );
    T->Element = TmpCell->Element;
    T->Right = Delete( T->Element, T->Right );
}
else /* One or zero children */
{
    TmpCell = T;
    if( T->Left == NULL ) /* Also handles 0 children */
        T = T->Right;
    else if( T->Right == NULL )
        T = T->Left;
    free( TmpCell );
}

return T;
}
```



Conclusion

- Generic trees terminology
 - ♦ Root, leaf, parent, child, sibling, ancestor, descendant, edge, path, depth, height, length
- Tree traversal
 - ♦ Preorder
 - ♦ Postorder
- Binary tree/complete binary tree
 - ♦ Binary tree traversal: Pre-, In-, Post-order
 - ♦ Recursive implementation
 - ♦ Expression tree
- Binary search tree and its operations
 - ♦ Find
 - ♦ Delete
 - ♦ Insert

Assignment 08

- Written assignment

Textbook: Exercise 4.1, 4.2, 4.3

- Submit it to Moodle.